

UNIT 2 – OBJECTS AND CLASSES

1. BASICS OF OBJECTS AND CLASSES

1.1 INTRODUCTION

Java is an **object-oriented programming language**. In Java, programs are mainly designed using **classes and objects**. A class describes the properties and actions that an object should have. An object is the actual entity created from a class. For example, `Student` can be a class, while `Rahul` and `Priya` can be objects of the `Student` class. Classes and objects help us organize large programs into smaller and manageable components.

1.2 CLASS

A **class** is a blueprint or template for creating objects. It defines the data and operations that an object can contain. A class can contain variables, methods, constructors and other members. The variables represent the **properties** of an object, while methods represent its **behaviour**. A class itself does not represent a particular individual object; it provides the structure from which objects are created.

SYNTAX

```
class ClassName {
    // variables
    // methods
}
```

EXAMPLE

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println(name);
        System.out.println(age);
    }
}
```

Here, `Student` is a class. `name` and `age` represent the properties of a student, while `display()` represents an action performed by the student object.

1.3 OBJECT

An **object** is an instance of a class. It represents a real-world entity and contains its own data. Objects are created using the `new` keyword in Java. Multiple objects can be created from the same class, and each object can have different values for its variables. For example, `Student` is a class, while `student1` and `student2` can be two different objects.

SYNTAX

```
ClassName objectName = new ClassName();
```

EXAMPLE

```
Student s1 = new Student();
Student s2 = new Student();
```

Here, s1 and s2 are two separate objects of the Student class.

1.4 CLASS AND OBJECT RELATIONSHIP

A class and an object are closely related, but they are not the same. A **class defines the structure**, while an **object represents an actual instance** of that structure. One class can be used to create many objects. Each object can contain different values for the same variables. This concept allows Java programs to represent real-world entities easily.

REAL-TIME EXAMPLE

Consider a college:

- **Class:** Student
- **Objects:** student1, student2, student3
- **Properties:** name, age, register number
- **Behaviour:** study(), attendClass(), writeExam()

The class provides the common structure, while each student object contains individual information.

1.5 CREATING AN OBJECT

Objects are generally created using the `new` keyword. The `new` keyword allocates memory for the object and returns a reference to it. The reference is stored in a variable so that we can access the object's members. Once the object is created, its variables and methods can be accessed using the **dot (.) operator**.

EXAMPLE

```
Student s1 = new Student();

s1.name = "Rahul";
s1.age = 20;

s1.display();
```

Here:

- Student → class name
- s1 → reference variable
- new Student() → creates the object
- s1.name → accesses the variable
- s1.display() → calls the method

1.6 COMPLETE PROGRAM

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Main {
    public static void main(String[] args) {

        Student s1 = new Student();

        s1.name = "Rahul";
        s1.age = 20;

        s1.display();
    }
}
```

OUTPUT

```
Name: Rahul
Age: 20
```

PROGRAM EXPLANATION

The `Student` class contains two variables, `name` and `age`, and one method called `display()`. Inside `main()`, the statement `new Student()` creates a `Student` object. The reference `s1` is used to access that object. Values are assigned to `name` and `age` using the dot operator. Finally, `display()` is called to print the student information.

2. CONSTRUCTORS

2.1 DEFINITION

A **constructor** is a special member of a class that is used to initialize an object when it is created. A constructor has the **same name as the class** and does not have a return type. It is automatically called when an object is created using the `new` keyword. Constructors are mainly used to give initial values to object variables. A class can have more than one constructor with different parameters.

2.2 FEATURES OF CONSTRUCTORS

The important features of constructors are:

1. A constructor has the **same name as the class**.
2. It does not have a return type.
3. It is automatically called when an object is created.
4. It is mainly used for **initializing objects**.
5. A class can have multiple constructors.
6. Constructors can be **overloaded**.
7. If no constructor is written, Java may provide a default constructor under appropriate conditions.

2.3 NO-ARGUMENT CONSTRUCTOR

A constructor that does not receive any arguments is called a **no-argument constructor**. It can be written by the programmer to initialize an object with fixed or default values. It is called automatically when an object is created without passing values. This type of constructor is useful when every object initially requires the same basic setup.

EXAMPLE

```
class Student {
    Student() {
        System.out.println("Student object created");
    }
}

class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
    }
}
```

OUTPUT

```
Student object created
```

2.4 PARAMETERIZED CONSTRUCTOR

A **parameterized constructor** is a constructor that accepts one or more parameters. It is used when we want to initialize an object with different values at the time of object creation. The values are passed inside the parentheses while creating the object. This makes it convenient to create multiple objects with different information.

EXAMPLE

```
class Student {
    String name;
    int age;
    Student(String n, int a) {
        name = n;
        age = a;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul", 20);
        Student s2 = new Student("Priya", 19);
        s1.display();
        s2.display();
    }
}
```

OUTPUT

```
Name: Rahul
Age: 20

Name: Priya
Age: 19
```

2.5 CONSTRUCTOR OVERLOADING

When a class contains **more than one constructor with different parameter lists**, it is called constructor overloading. The constructors must differ in the number or type of parameters. Java determines which constructor to execute based on the arguments passed during object creation. Constructor overloading allows objects to be initialized in different ways.

EXAMPLE

```
class Student {

    Student() {
        System.out.println("No details available");
    }

    Student(String name) {
        System.out.println("Name: " + name);
    }

    Student(String name, int age) {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Main {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student("Rahul");
        Student s3 = new Student("Priya", 20);
    }
}
```

2.6 CONSTRUCTOR VS METHOD

CONSTRUCTOR	METHOD
Used mainly to initialize objects	Used to perform an operation
Same name as class	Can have any valid name
Has no return type	Can have a return type
Called automatically during object creation	Usually called explicitly
Cannot be inherited like normal methods	Methods can participate in inheritance
Can be overloaded	Can also be overloaded

3. FINALIZE

3.1 MEANING OF FINALIZE()

`finalize()` is a method that was historically associated with **garbage collection** in Java. When an object was no longer reachable, it could become eligible for garbage collection. Historically, Java allowed `finalize()` to be associated with cleanup before an object was reclaimed. However, modern Java has deprecated finalization because it is unreliable and has safer resource-management mechanisms. For your syllabus, students mainly need to understand its basic relationship with objects and garbage collection.

3.2 GARBAGE COLLECTION

Garbage collection is the automatic memory-management process in Java. When an object is no longer reachable by the program, its memory can eventually be reclaimed by the Java Virtual Machine. The programmer does not normally need to manually release object memory as in languages such as C. The garbage collector works automatically in the background. This helps reduce memory-management errors in Java programs.

EXAMPLE

```
Student s1 = new Student();

s1 = null;
```

After `s1 = null`, if there is no other reference to that `Student` object, the object can become **eligible for garbage collection**.

3.3 FINALIZE() METHOD

The traditional form of the method is:

```
protected void finalize() {
    // cleanup code
}
```

Historically, the method was intended to perform cleanup before an object was removed by garbage collection. However, programmers could not rely on exactly when, or even whether, it would execute. Therefore, modern Java discourages its use. It is important to understand the concept because it appears in many older Java textbooks and syllabi.

3.4 EXAMPLE

```
class Demo {

    protected void finalize() {
        System.out.println("Finalize method called");
    }

    public static void main(String[] args) {

        Demo d = new Demo();
        d = null;

        System.gc();
    }
}
```

IMPORTANT

Do **not** teach students that `System.gc()` guarantees that `finalize()` will execute. It does not. The example is only for understanding the historical concept.

3.5 IMPORTANT POINTS

- `finalize()` was associated with garbage collection.
- It was intended for cleanup before object reclamation.
- It has no return value.
- It was not normally called directly by the programmer.
- The JVM controlled when garbage collection occurred.
- Execution of `finalize()` was never something a program should depend upon.
- `finalize()` is **deprecated in modern Java**.

3.6 FINAL, FINALLY AND FINALIZE()

These three terms look similar but have completely different purposes.

TERM	MEANING
<code>final</code>	Keyword used to prevent modification/inheritance/overriding in specific contexts
<code>finally</code>	Block used with exception handling
<code>finalize()</code>	Legacy method associated with garbage collection; deprecated

4. VISIBILITY MODIFIERS

Absolutely. For **Visibility Modifiers**, each modifier should have its **own explanation + syntax + separate simple program + output + explanation**. That is much better for teaching BCA students.

4. VISIBILITY MODIFIERS IN JAVA**4.1 INTRODUCTION**

Visibility modifiers, also called **access modifiers**, are keywords used to control the accessibility of classes, variables, methods and constructors in Java. They decide **where a particular member can be accessed** from. Java provides four access levels: `private`, `default`, `protected`, and `public`. These modifiers help in achieving **data hiding and encapsulation**. By using the appropriate modifier, we can prevent unwanted access to important data.

FOUR VISIBILITY MODIFIERS

1. **private**
2. **Default**
3. **protected**
4. **public**

4.2 PRIVATE MODIFIER

DEFINITION

The `private` modifier provides the **most restricted access** to a member. A private variable or method can be accessed directly **only inside the same class** in which it is declared. It cannot be accessed directly from another class. Private members are commonly used to protect important data from direct modification. This is an important part of **encapsulation** in Java.

SYNTAX

```
private dataType variableName;
```

EXAMPLE PROGRAM

```
class Student {

    private int marks = 85;

    void display() {
        System.out.println("Marks: " + marks);
    }
}

class Main {
    public static void main(String[] args) {

        Student s = new Student();

        s.display();
    }
}
```

OUTPUT

```
Marks: 85
```

EXPLANATION

- `marks` is declared as `private`.
- Therefore, it can be directly accessed inside the `Student` class.
- The `display()` method belongs to the same class.
- So, `display()` can access `marks`.
- We cannot write `s.marks` directly inside `Main`.
- This protects the variable from direct outside access.

THIS IS NOT ALLOWED

```
System.out.println(s.marks);
```

because `marks` is `private`.

4.3 DEFAULT MODIFIER

DEFINITION

When no access modifier is written before a variable, method or class member, it has **default access**. Default members can be accessed by other classes **within the same package**. They cannot normally be accessed directly from classes in a different package. The keyword `default` is not written explicitly. Default access is also called **package-private access**.

SYNTAX

```
dataType variableName;
```

There is **no keyword** before the variable.

EXAMPLE PROGRAM

```
class Student {
    int age = 20;

    void display() {
        System.out.println("Age: " + age);
    }
}

class Main {
    public static void main(String[] args) {

        Student s = new Student();

        System.out.println("Student Age: " + s.age);
        s.display();
    }
}
```

OUTPUT

```
Student Age: 20
Age: 20
```

EXPLANATION

- age **does not have** `private`, `protected`, or `public`.
- Therefore, it has **default access**.
- Main can access age because both classes are in the same package in this simple example.
- The `display()` method also has default access.
- Default access is useful when classes within the same package need to share information.

IMPORTANT POINT

```
int age = 20;
```

means **default access**.

It does **not** mean:

```
default int age = 20;    // Wrong
```

4.4 PROTECTED MODIFIER

DEFINITION

The `protected` modifier allows a member to be accessed within the **same package**. It also allows a subclass in another package to access the protected member through inheritance, subject to Java's access rules. It is mainly useful when working with **inheritance**. A protected member provides more access than `private` but less general access than `public`.

SYNTAX

```
protected dataType variableName;
```

EXAMPLE PROGRAM

```
class Student {
    protected int marks = 90;
}

class Main extends Student {
    void display() {
        System.out.println("Marks: " + marks);
    }

    public static void main(String[] args) {
        Main obj = new Main();
        obj.display();
    }
}
```

OUTPUT

```
Marks: 90
```

EXPLANATION

- `marks` is declared using `protected`.
- `Main` extends `Student`.
- Therefore, `Main` becomes a subclass of `Student`.
- The subclass can access the protected `marks`.
- The `display()` method prints the value.
- This demonstrates the use of `protected` with inheritance.

REAL-TIME EXAMPLE

Suppose we have a general `Employee` class containing employee information. Classes such as `Manager` and `Developer` can inherit from `Employee`. Some information can be declared `protected` so that subclasses can use it.

```
Employee
  ↓
┌───┴───┐
Manager Developer
```

4.5 PUBLIC MODIFIER

DEFINITION

The `public` modifier provides the **widest accessibility** among Java's access modifiers. A public member can generally be accessed from other classes and packages, provided the containing class is accessible. Public methods are often used to provide services or functionality to other classes. However, sensitive data should not automatically be made public because doing so removes access restrictions.

SYNTAX

```
public dataType variableName;
```

EXAMPLE PROGRAM

```
class Student {

    public String name = "Rahul";

    public void display() {
        System.out.println("Student Name: " + name);
    }
}

class Main {
    public static void main(String[] args) {

        Student s = new Student();

        System.out.println(s.name);

        s.display();
    }
}
```

OUTPUT

```
Rahul
Student Name: Rahul
```

EXPLANATION

- `name` is declared as `public`.
- Therefore, it can be accessed from the `Main` class.
- `display()` is also declared as `public`.
- The object `s` is used to access both members.
- Public members have the widest accessibility.
- This is useful when a class needs to expose certain functionality to other classes.

REAL-TIME EXAMPLE

A `Student` class may provide a public method such as:

```
public void displayDetails()
```

Other parts of a college application can call this method to display student information.

5. METHODS AND OBJECTS

5.1 INTRODUCTION TO METHODS

A **method** is a named block of statements that performs a particular task in a Java program. Methods help us divide a large program into smaller and meaningful parts. Instead of writing the same statements repeatedly, we can write them once inside a method and call the method whenever required. A method can receive data from the calling program through parameters. A method can also return a result to the calling program.

EXAMPLE

Consider a calculator application. Instead of writing all calculations inside `main()`, we can create separate methods:

```
add()
subtract()
multiply()
divide()
```

Each method performs one specific operation.

ADVANTAGES OF METHODS

1. **Code Reusability** – The same method can be called many times.
2. **Easy to Understand** – A large program can be divided into smaller sections.
3. **Easy Debugging** – Errors can be located more easily.
4. **Reduces Repetition** – The same code does not have to be written again.
5. **Easy Maintenance** – Changes can be made inside a particular method without affecting the entire program.

5.2 SYNTAX OF A METHOD

The general syntax of a method is:

```
returnType methodName(parameterList) {
    // statements
}
```

EXAMPLE

```
void display() {
    System.out.println("Hello Java");
}
```

Here:

- `void` → return type
- `display` → method name
- `()` → parameter list
- `{ }` → method body

5.3 PARTS OF A METHOD

A method can contain several important parts.

```
int add(int a, int b) {
    return a + b;
}
```

1. RETURN TYPE

`int` is the return type. It tells us that the method returns an integer value.

2. METHOD NAME

`add` is the name of the method. It should be a meaningful name that describes what the method does.

3. PARAMETERS

`int a, int b` are parameters. They receive values from the calling statement.

4. METHOD BODY

The statements inside `{ }` form the method body.

5. RETURN STATEMENT

`return a + b;` sends the calculated value back to the calling statement.

5.4 METHOD WITHOUT PARAMETERS AND RETURN VALUE

A method does not always need to receive input or return a result. A method can simply perform an operation.

EXAMPLE

```
class Student {
    void display() {
        System.out.println("Welcome to BCA");
    }
}

class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

OUTPUT

Welcome to BCA

EXPLANATION

The `display()` method does not accept any parameter. It also does not return any value because its return type is `void`. The method is called using the object `s`.

5.5 METHOD WITH PARAMETERS

A method can accept values from the calling statement. These values are called **arguments**, while the variables that receive them are called **parameters**.

EXAMPLE

```
class Student {
    void display(String name, int age) {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display("Rahul", 20);
    }
}
```

OUTPUT

```
Name: Rahul
Age: 20
```

EXPLANATION

The method has two parameters:

```
String name
int age
```

When the method is called:

```
s.display("Rahul", 20);
```

the value "Rahul" is passed to `name` and 20 is passed to `age`.

5.6 METHOD WITH RETURN VALUE

A method can calculate something and send the result back to the calling program. Such a method uses a return type other than `void`. The `return` keyword is used to send the result.

EXAMPLE

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}

class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        int result = c.add(10, 20);
        System.out.println("Result = " + result);
    }
}
```

OUTPUT

Result = 30

WORKING

```
10 + 20
  ↓
add() method
  ↓
30 returned
  ↓
result
  ↓
Output
```

5.7 CALLING A METHOD USING AN OBJECT

An **instance method** is generally called using an object of the class.

SYNTAX

```
objectName.methodName();
```

EXAMPLE

```
Student s = new Student();

s.display();
```

5.8 MULTIPLE OBJECTS AND METHODS

One class can create multiple objects. Each object can contain different values, but the objects can use the same methods.

EXAMPLE

```
class Student {
    String name;

    void display() {
        System.out.println("Name: " + name);
    }
}
class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();

        s1.name = "Rahul";
        s2.name = "Priya";

        s1.display();
        s2.display();
    }
}
```

OUTPUT

```
Name: Rahul
Name: Priya
```

IMPORTANT CONCEPT

The method `display()` is defined only once, but both objects can use it.

5.9 METHODS AND OBJECTS – EXAMPLE

Consider an **Employee Management System**. We can create:

Class → Employee

Data:

```
name
salary
department
```

Methods:

```
displayDetails()
calculateSalary()
```

Objects can be:

```
employee1
employee2
employee3
```

Each employee object has different data but can use the same methods.

6. BUILT-IN CLASSES IN JAVA

6.1 INTRODUCTION

Java provides many ready-made classes through its **Java API**. These are called built-in or predefined classes. Programmers can use these classes instead of writing common functionality from the beginning. They provide methods for working with strings, characters, files, collections, dates and many other operations. In this unit, we study four important built-in classes: **String**, **Character**, **StringBuffer** and **File**.

6.2 STRING CLASS

DEFINITION

The **String class** is used to represent a sequence of characters. Strings are one of the most commonly used data types in Java programs. Names, addresses, messages, email IDs and passwords are examples of information represented using strings. Java provides many methods for searching, comparing, extracting and modifying string representations. The `String` class belongs to the `java.lang` package, so it does not normally need to be imported.

6.2.1 CREATING A STRING

USING STRING LITERAL

```
String name = "Vijay";
```

This is the most common way to create a `String`.

USING NEW

```
String name = new String("Vijay");
```

Both create `String` objects, but the first form is generally preferred for ordinary string literals.

6.2.2 STRING IS IMMUTABLE

A `String` is **immutable**, which means that once a `String` object is created, its contents cannot be changed. For example:

```
String name = "Java";
name = name + " Programming";
```

Here, the original `"Java"` `String` is not modified. A new `String` value is created for `"Java Programming"` and the variable `name` refers to it.

SIMPLE IDEA

```
"Java"
  ↓
String object
```

```
name = name + " Programming"
  ↓
"Java Programming"
  ↓
new String value
```

Students should remember: **String is immutable.**

6.2.3 LENGTH ()

The `length()` method returns the number of characters in a String.

EXAMPLE

```
class Main {
    public static void main(String[] args) {
        String name = "Rahul";
        System.out.println(name.length());
    }
}
```

OUTPUT

5

There are five characters:

```
R a h u l
1 2 3 4 5
```

6.2.4 CHARAT ()

The `charAt()` method returns the character at a specified index.

Java uses **zero-based indexing**.

```
R a h u l
0 1 2 3 4
```

EXAMPLE

```
class Main {
    public static void main(String[] args) {
        String name = "Rahul";
        System.out.println(name.charAt(0));
        System.out.println(name.charAt(3));
    }
}
```

OUTPUT

```
R
u
```

6.2.5 EQUALS ()

The `equals()` method compares the contents of two strings.

EXAMPLE

```
class Main {
    public static void main(String[] args) {

        String a = "Java";
        String b = "Java";

        System.out.println(a.equals(b));
    }
}
```

OUTPUT

true

If the contents are different:

```
String a = "Java";
String b = "Python";
```

the result will be:

```
false
```

6.2.6 TOUPPERCASE ()

This method converts a String into uppercase letters.

```
String name = "java";

System.out.println(name.toUpperCase());
```

OUTPUT

```
JAVA
```

6.2.7 TOLOWERCASE ()

This method converts a String into lowercase letters.

```
String name = "JAVA";

System.out.println(name.toLowerCase());
```

OUTPUT

```
Java
```

6.2.8 SUBSTRING ()

The `substring()` method is used to extract a portion of a String.

EXAMPLE

```
class Main {
    public static void main(String[] args) {

        String name = "Computer";

        System.out.println(name.substring(0, 4));
    }
}
```

OUTPUT

```
Comp
```

The ending index is not included.

6.2.9 CONCAT ()

The `concat ()` method joins two strings.

```
class Main {
    public static void main(String[] args) {

        String first = "Hello ";
        String second = "Java";

        System.out.println(first.concat(second));
    }
}
```

OUTPUT

Hello Java

6.2.10 IMPORTANT STRING METHODS

METHOD	PURPOSE
<code>length ()</code>	Finds number of characters
<code>charAt ()</code>	Returns character at specified index
<code>equals ()</code>	Compares String contents
<code>substring ()</code>	Extracts part of a String
<code>concat ()</code>	Joins strings
<code>toUpperCase ()</code>	Converts to uppercase
<code>toLowerCase ()</code>	Converts to lowercase
<code>trim ()</code>	Removes leading/trailing whitespace

6.3 CHARACTER CLASS

DEFINITION

The **Character class** is a predefined Java class used to work with individual characters. It provides several useful methods for checking and converting characters. For example, we can determine whether a character is a letter, digit, uppercase letter or lowercase letter. The Character class belongs to the `java.lang` package. It is particularly useful when processing characters individually.

6.3.1 ISLETTER()

The `isLetter()` method checks whether a character is a letter.

EXAMPLE

```
class Main {  
    public static void main(String[] args) {  
  
        char ch = 'A';  
  
        System.out.println(Character.isLetter(ch));  
    }  
}
```

OUTPUT

true

6.3.2 ISDIGIT()

The `isDigit()` method checks whether a character is a numeric digit.

EXAMPLE

```
class Main {  
    public static void main(String[] args) {  
        char ch = '5';  
        System.out.println(Character.isDigit(ch));  
    }  
}
```

OUTPUT

true

6.3.3 ISUPPERCASE()

This method checks whether a character is uppercase.

```
char ch = 'A';  
  
System.out.println(Character.isUpperCase(ch));
```

OUTPUT

true

6.3.4 ISLOWERCASE()

This method checks whether a character is lowercase.

```
char ch = 'a';  
  
System.out.println(Character.isLowerCase(ch));
```

OUTPUT

True

6.3.5 CHARACTER CONVERSION

Character methods can also convert characters from uppercase to lowercase and vice versa.

```
class Main {
    public static void main(String[] args) {

        char ch = 'A';

        System.out.println(Character.toLowerCase(ch));
    }
}
```

OUTPUT

a

6.3.6 CHARACTER CLASS – REAL-TIME EXAMPLE

Suppose a registration form asks the user to enter a mobile number. We can use `isDigit()` to check whether every entered character is a digit.

Similarly, `isLetter()` can be used to check whether a name contains letters.

6.4 STRINGBUFFER CLASS

DEFINITION

The **StringBuffer** class is used to represent a sequence of characters that can be modified. Unlike `String`, `StringBuffer` is **mutable**. This means we can change its contents without creating a completely new `String` object for every modification. `StringBuffer` provides methods such as `append()`, `insert()`, `delete()` and `reverse()`. It is useful when a program performs several modifications on character data.

6.4.1 CREATING STRINGBUFFER

```
StringBuffer sb = new StringBuffer("Hello");
```

Here, `sb` is a `StringBuffer` object containing "Hello".

6.4.2 APPEND ()

The `append()` method adds data to the end of the existing `StringBuffer`.

EXAMPLE

```
class Main {
    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" Java");
        System.out.println(sb);
    }
}
```

OUTPUT

Hello Java

6.4.3 INSERT ()

The `insert()` method inserts data at a specified position.

```
class Main {
    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("Hello");

        sb.insert(5, " Java");

        System.out.println(sb);
    }
}
```

OUTPUT

Hello Java

6.4.4 DELETE ()

The `delete()` method removes characters from a specified range.

```
class Main {
    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("Hello Java");

        sb.delete(5, 10);

        System.out.println(sb);
    }
}
```

OUTPUT

Hello

6.4.5 REVERSE ()

The `reverse()` method reverses the characters.

```
class Main {
    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("Java");

        sb.reverse();

        System.out.println(sb);
    }
}
```

OUTPUT

avaJ

6.4.6 STRING VS STRINGBUFFER

STRING	STRINGBUFFER
Immutable	Mutable
Contents cannot be directly changed	Contents can be modified
Suitable for normal text	Useful for repeated modifications
Methods produce new String results where applicable	Methods modify the buffer

EASY EXAMPLE

String
"Hello" → cannot directly change existing object

StringBuffer
"Hello" → append → "Hello Java"

6.5 FILE CLASS

DEFINITION

The **File class** is a predefined class in the `java.io` package. It is used to represent the pathname of a file or directory. It provides methods for checking whether a file exists, obtaining its name, finding its size and determining whether a path represents a file or directory. The File class can also be used for some file and directory operations. However, it does not itself provide general-purpose methods for reading the actual contents of a file.

6.5.1 IMPORTING FILE CLASS

Since File belongs to `java.io`, we generally write:

```
import java.io.File;
```

6.5.2 CREATING A FILE OBJECT

```
File f = new File("test.txt");
```

This creates a File object representing the path `test.txt`. It does **not necessarily create the physical file**.

6.5.3 EXISTS()

The `exists()` method checks whether the specified file or directory exists.

EXAMPLE

```
import java.io.File;

class Main {
    public static void main(String[] args) {

        File f = new File("test.txt");

        System.out.println(f.exists());
    }
}
```

If the file exists:

true

If it does not exist:

false

6.5.4 GETNAME ()

The `getName()` method returns the name of the file or directory.

```
import java.io.File;

class Main {
    public static void main(String[] args) {

        File f = new File("test.txt");

        System.out.println(f.getName());
    }
}
```

OUTPUT

test.txt

6.5.5 LENGTH ()

The `length()` method returns the size of a file in **bytes**.

```
import java.io.File;

class Main {
    public static void main(String[] args) {

        File f = new File("test.txt");

        System.out.println("File Size: " + f.length() + " bytes");
    }
}
```

The exact output depends on the actual file size.

6.5.6 ISFILE ()

The `isFile()` method checks whether the specified path represents a normal file.

```
import java.io.File;

class Main {
    public static void main(String[] args) {

        File f = new File("test.txt");

        System.out.println(f.isFile());
    }
}
```

If test.txt is a file: true

6.5.7 ISDIRECTORY()

The `isDirectory()` method checks whether the specified path represents a directory.

```
import java.io.File;

class Main {
    public static void main(String[] args) {

        File f = new File("Documents");

        System.out.println(f.isDirectory());
    }
}
```

If Documents is a directory:

True

6.5.8 IMPORTANT FILE METHODS

METHOD	PURPOSE
<code>exists()</code>	Checks whether path exists
<code>getName()</code>	Returns file/directory name
<code>getPath()</code>	Returns path
<code>getAbsolutePath()</code>	Returns absolute path
<code>length()</code>	Returns file size
<code>isFile()</code>	Checks whether it is a file
<code>isDirectory()</code>	Checks whether it is a directory
<code>mkdir()</code>	Creates a directory
<code>delete()</code>	Deletes a file or empty directory

7. THIS REFERENCE

7.1 DEFINITION

The **this** keyword in Java is a reference that refers to the **current object**. It is mainly used inside instance methods and constructors. When local variables or parameters have the same name as instance variables, **this** helps to distinguish between them. It can also be used to call another method or constructor of the current class. For BCA students, the most important use is understanding **this** with instance variables and constructors.

7.2 WHY DO WE USE THIS?

The most common use of **this** is to differentiate between an **instance variable** and a **method/constructor parameter** having the same name.

For example:

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}
```

Here:

- `this.name` → instance variable
- `name` → constructor parameter
- `this` → current Student object

Without **this**, Java would treat both `name` references as the parameter.

7.3 SIMPLE EXAMPLE

```
class Student {
    String name;
    int age;
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void display() {
        System.out.println("Name: " + this.name);
        System.out.println("Age: " + this.age);
    }
}
class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul", 20);
        s1.display();
    }
}
```

OUTPUT

```
Name: Rahul
Age: 20
```

7.4 HOW THIS WORKS

When we create:

```
Student s1 = new Student("Rahul", 20);
```

the constructor receives:

```
name = "Rahul"
age  = 20
```

Then:

```
this.name = name;
this.age = age;
```

means:

```
current object's name = parameter name
current object's age  = parameter age
```

So the values are stored in the object `s1`.

7.5 THIS TO CALL CURRENT OBJECT'S METHOD

`this` can also be used to refer to the current object's method.

```
class Student {
    void display() {
        System.out.println("Student Details");
    }

    void show() {
        this.display();
    }
}

class Main {
    public static void main(String[] args) {
        Student s = new Student();

        s.show();
    }
}
```

OUTPUT

```
Student Details
```

Here:

```
this.display();
```

means **call the `display()` method of the current object.**

7.6 IMPORTANT POINTS

- `this` refers to the **current object**.
- It is mainly used inside instance methods and constructors.
- It helps distinguish instance variables from parameters.
- `this.variable` refers to the current object's variable.
- `this.method()` calls the current object's method.
- It is very commonly used in constructors.

EASY EXAMPLE TO REMEMBER

```
Student(String name) {  
    this.name = name;  
}
```

`this.name` = object's variable

`name` = parameter

UNIT 2 – IMPORTANT QUESTIONS**2 MARKS**

1. Define a class in Java.
2. Define an object in Java.
3. What is the use of the `new` keyword?
4. Define a constructor.
5. State any two characteristics of a constructor.
6. What is a parameterized constructor?
7. What is constructor overloading?
8. Define a method in Java.
9. What is `this` reference?
10. Mention any two uses of `this` keyword.
11. What are visibility modifiers?
12. Name the four visibility modifiers in Java.
13. What is the use of `private` modifier?
14. What is the use of `protected` modifier?
15. What is the use of `public` modifier?
16. What is default access in Java?
17. What is `finalize()` method?
18. What is garbage collection?
19. Define `StringBuffer`.
20. Define `File` class.

5 MARKS

1. **Define class and object with a suitable example.**
2. **What is a constructor? Explain its characteristics.**
3. Explain different types of constructors in Java.
4. **Explain constructor overloading with a suitable example.**
5. **Explain different visibility modifiers in Java.**
6. Explain `private`, `public` and `protected` modifiers with examples.
7. Explain methods with parameters and return values with examples.
8. **Explain `this` reference with a suitable example.**
9. Write a short note on `String` class and its important methods.
10. Write a short note on `File` class and its important methods.

10 MARKS

1. **With a suitable example, explain how to declare and define a class and object in Java.**
2. **What is a constructor? Explain different types of constructors with suitable examples.**
3. **Write a note on constructor overloading with a suitable Java program.**
4. **Explain the different visibility/access modifiers in Java with suitable examples.**
5. Explain methods and objects in Java with suitable examples.